

RDKDC Lab 3 Report

Anishka Bhartiya, Lujia Yang, Xiangzheng Fang

April 15, 2026

1 Task 2: Resolved Rate Control

1.1 UR5 & UR5e Parameters

The geometric parameters for both robot types (derived from Fig. 1 and Fig. 2) are defined as follows. Note that for calculations, we use the specific offsets (e.g., L_0) to define the twist axes in the base frame.

Type	L_0 (m)	L_1 (m)	L_2 (m)	L_3 (m)	L_4 (m)	L_5 (m)
UR5	0.08916	0.425	0.39225	0.10915	0.09465	0.0823
UR5e	0.1625	0.425	0.3922	0.1333	0.0997	0.0996

Table 1: Link length definitions for UR5 and UR5e used in MATLAB implementation.

Joint Twists The joint twists $\xi_i = [v_i; \omega_i]$ are constructed based on the axis of rotation ω_i and a point q_i on the axis. Based on the schematic provided:

- $\xi_1 = [0, 0, 0, 0, 0, 1]^T$ (Base rotation)
- $\xi_2 = [-L_0, 0, 0, 0, 0, 1]^T$
- $\xi_3 = [-L_0, 0, L_1, 0, 0, 1]^T$
- $\xi_4 = [-L_0, 0, L_1 + L_2, 0, 0, 1]^T$
- $\xi_5 = [-L_3, L_1 + L_2, 0, 0, 0, -1]^T$
- $\xi_6 = [0.00549, 0, L_1 + L_2, 0, 0, 1]^T$

Reference Configuration The initial end-effector pose $g_{st}(0)$ represents the transformation from the base frame to `tool0` when all joint angles are zero:

$$g_{st}(0) = \begin{bmatrix} -1 & 0 & 0 & 0.8173 \\ 0 & 0 & 1 & 0.1914 \\ 0 & 1 & 0 & -0.0055 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1.2 urFwdKin()

The forward kinematics map $g_{st}(\theta)$ is implemented using the Product of Exponentials (PoE) formula in the spatial frame:

$$g_{st}(\theta) = e^{\hat{\xi}_1\theta_1} e^{\hat{\xi}_2\theta_2} e^{\hat{\xi}_3\theta_3} e^{\hat{\xi}_4\theta_4} e^{\hat{\xi}_5\theta_5} e^{\hat{\xi}_6\theta_6} g_{st}(0)$$

where $\hat{\xi}_i \in se(3)$ is the 4×4 matrix representation of the twist.

1.3 urBodyJacobian()

The Body Jacobian $J_{st}^b(q)$ relates the joint velocities to the twist in the end-effector frame. In our implementation, we first compute the spatial Jacobian J^s and then transform it to the body frame:

$$J_{st}^b(q) = Ad_{g_{st}(q)}^{-1} J_{st}^s(q)$$

The i^{th} column of the spatial Jacobian is given by $J_{si} = Ad_{e^{\hat{\xi}_1\theta_1} \dots e^{\hat{\xi}_{i-1}\theta_{i-1}}} \hat{\xi}_i$.

1.4 manipulability()

We implemented three measures to evaluate the robot's proximity to singularities:

1. **Minimum Singular Value:** $\mu_1 = \sigma_{min}(J)$, indicating the "worst-case" dexterity.
2. **Determinant:** $\mu_2 = \sqrt{\det(JJ^T)}$, proportional to the volume of the manipulability ellipsoid.
3. **Inverse Condition Number:** $\mu_3 = \frac{\sigma_{min}(J)}{\sigma_{max}(J)}$, representing the sensitivity to joint errors.

1.5 getXi()

The function `getXi.m` implements the matrix logarithm for $SE(3)$. Given $g = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$, we find $\hat{\xi} = \log(g)$ such that:

- ω is extracted from the rotation matrix R using the axis-angle representation $\theta = \cos^{-1}\left(\frac{tr(R)-1}{2}\right)$.
- v is solved via $v = G^{-1}(\theta)p$, where $G(\theta) = I + \frac{1-\cos\theta}{\theta^2}\hat{\omega} + \frac{\theta-\sin\theta}{\theta^3}\hat{\omega}^2$ [cite: 111].

1.6 urRRcontrol()

The Resolved-Rate control system iteratively updates the joint configuration to minimize the error between the current pose g_{st} and the desired pose g_{st}^* :

$$q_{k+1} = q_k - KT_{step} [J_{st}^b(q_k)]^{-1} \xi_k$$

where the error twist ξ_k is computed from the error matrix $g_{err} = g_{st}^{-1} g_{st}^*$. The controller terminates when $\|v_k\| < 0.5\text{cm}$ and $\|\omega_k\| < 2^\circ$. If the system approaches a singularity ($\sigma_{min} < 0.01$), the script returns -1 to ensure safety.

2 Task 2: ROS Validation

2.1 Forward Kinematics Verification

The purpose of this test is to verify the accuracy of the `urFwdKin` function by comparing the calculated end-effector transformation matrix g_{st} with the actual pose reported by the robot interface in RVIZ. We performed two sets of tests with different joint configurations to ensure consistency.

2.1.1 Test Configuration and Results

Two distinct joint vectors, q_1 and q_2 , were selected to test the robot in non-singular and general poses. For each test, the analytical transformation g_{calc} was computed using the implemented forward kinematics, and the reported transformation $g_{reported}$ was obtained via `ur.get_current_transformation`.

- **Test Case 1:** $q_1 = [0.5, -1.0, 1.2, -0.2, 1.5, 0.3]^T$ (rad)
- **Test Case 2:** $q_2 = [0.2, -1.2, 0.5, 0.1, 1.4, 0.2]^T$ (rad)

The comparison results are summarized in Table 2. The error is quantified using the Frobenius Norm of the difference between the two matrices: $E = \|g_{calc} - g_{reported}\|_F$.

Table 2: Forward Kinematics Verification Results

Test Case	Robot Model	Pose Error (Frobenius Norm)
Test 1 (q_1)	UR5e	2.949503×10^{-2}
Test 2 (q_2)	UR5e	2.018×10^{-15}

2.1.2 Visualization

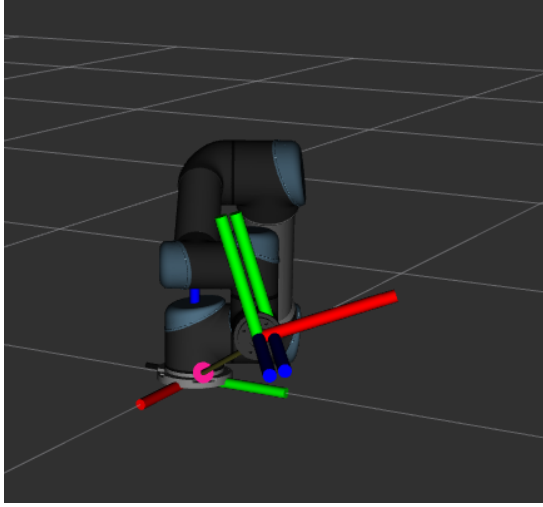
To visually confirm the forward kinematics implementation, we utilized the RVIZ simulation environment. As illustrated in Figure 1, for each test case, a coordinate frame labeled `fwdKinToolFrame` was programmatically generated at the calculated pose g_{calc} .

2.2 Jacobian Verification via Central Difference

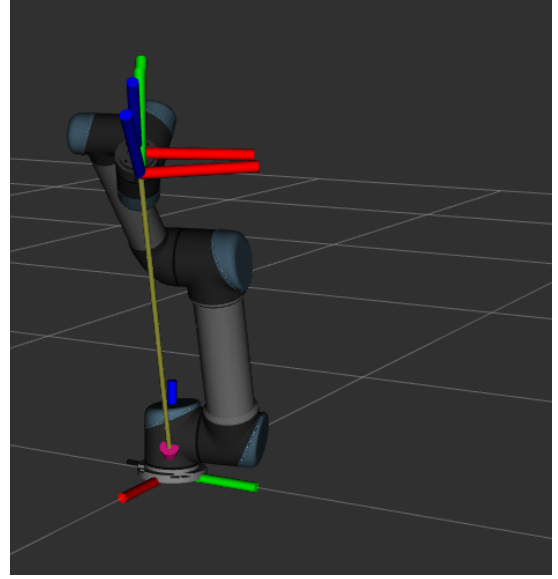
The body Jacobian implementation in `urBodyJacobian` was verified using a central-difference numerical approximation. For a joint configuration q , the i^{th} column of the approximate Jacobian J_{approx} is calculated by perturbing the i^{th} joint by a small $\epsilon = 10^{-4}$:

$$\xi_{i,approx} \approx \left(g_{st}(q)^{-1} \frac{g_{st}(q + \epsilon e_i) - g_{st}(q - \epsilon e_i)}{2\epsilon} \right)^v$$

Note that the resulting matrix was "twist-ified" by taking the skew-symmetric part of the upper-left 3×3 block to ensure it resides in $se(3)$. As shown in the terminal output (Figure 2), the matrix norm of the error between the analytical and numerical Jacobian is approximately 3.124×10^{-9} , which confirms the mathematical correctness of our Jacobian function.



(a) Test Case 1: q_1



(b) Test Case 2: q_2

Figure 1: RVIZ visualization comparing the calculated `fwdKinToolFrame` and the actual `tool0` frame. The perfect overlap in both (a) and (b) verifies the forward kinematics model.

```

--- Part (b): Testing Body Jacobian ---
Jacobian Error Norm: 3.124234e-09

```

Figure 2: MATLAB terminal output showing the Jacobian error norm.

2.3 Manipulability Analysis near Singularities

We evaluated the three manipulability measures implemented in `manipulability.m` as the robot approached a singular configuration. Specifically, we varied θ_3 from $-\pi/4$ to $\pi/4$ while keeping other joints at a configuration near singularity ($\theta_5 = 0$).

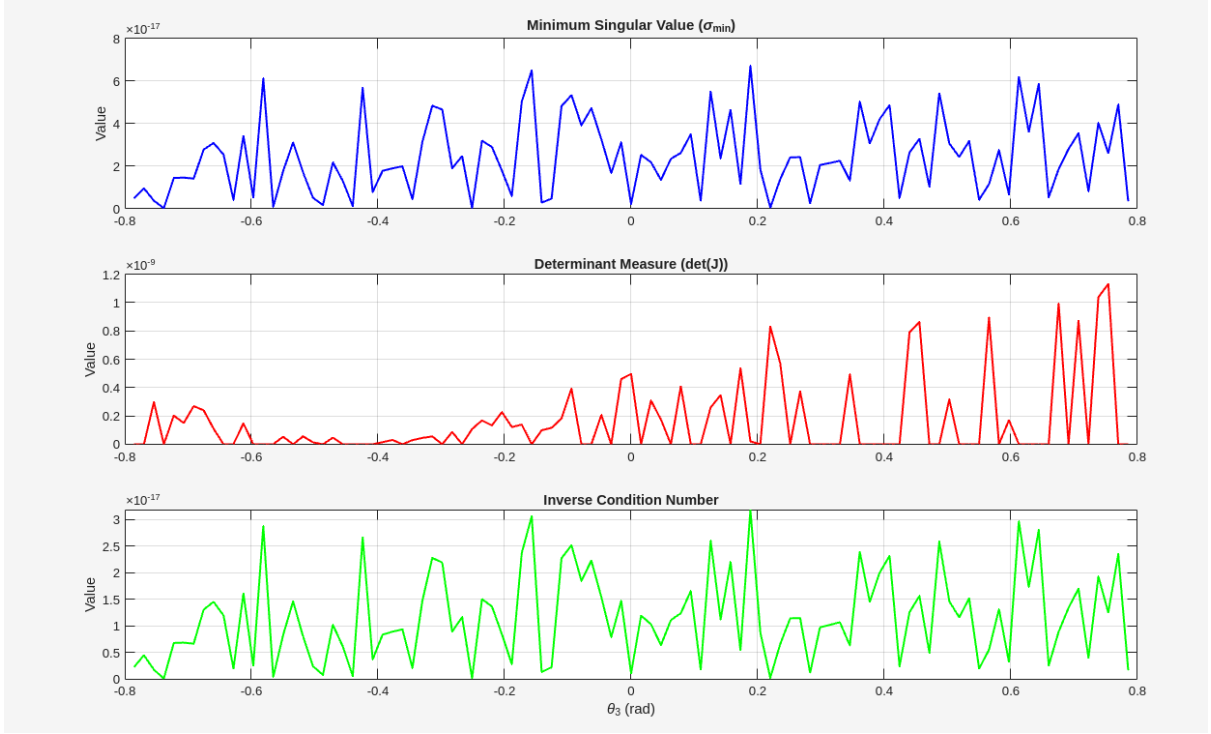


Figure 3: Comparison of manipulability measures (Minimum Singular Value, Determinant, and Inverse Condition Number) as a function of θ_3 .

As illustrated in Figure 3, all three measures accurately identify the singularity at $\theta_3 \approx 0$. The minimum singular value (σ_{min}) and the inverse condition number drop significantly as the robot loses a degree of freedom, providing a reliable metric for the controller to abort motion to prevent high joint velocities.

2.4 Resolved-Rate Control Performance

The `urRRcontrol` function was tested under two scenarios: normal convergence and singularity-induced safety abort.

1. **Normal Convergence:** Starting from a neutral pose, the controller successfully drove the end-effector to the desired goal g_{st^*} . As shown in Figure 4, the controller achieved a final positional error of 0.4977 cm, which is well within the required threshold of 0.5 cm.
2. **Singularity Abort:** To test safety robustness, the robot was commanded to move while in a singular configuration ($\theta_3 = 0, \theta_5 = 0$). The controller correctly detected the low manipulability ($\sigma_{min} < 0.01$), aborted the operation, and returned -1 as expected.

```

--- Part (e): Testing Resolved Rate Control ---
Phase 1: Testing normal convergence...

Convergence Successful! Final error: 0.4977 cm
Please wait a moment...

Phase 2: Testing Singularity Abort logic...
Moving robot to a singular configuration...

Attempting to control near singularity (expecting -1)...
Success: Controller correctly ABORTED and returned -1.

```

Figure 4: Resolved-Rate control test results for normal convergence and singularity abort logic.

3 Task 2: URSim Validation

In accordance with the lab handout, the RTDE interface was validated exclusively for Task 2 Part (e) using the URSim environment[cite: 185, 186]. This section ensures that the resolved-rate control logic implemented in MATLAB is robust across different communication protocols (ROS and RTDE).

3.1 Experimental Setup

The experiment was conducted using a local Docker-based URSim instance with the following configuration:

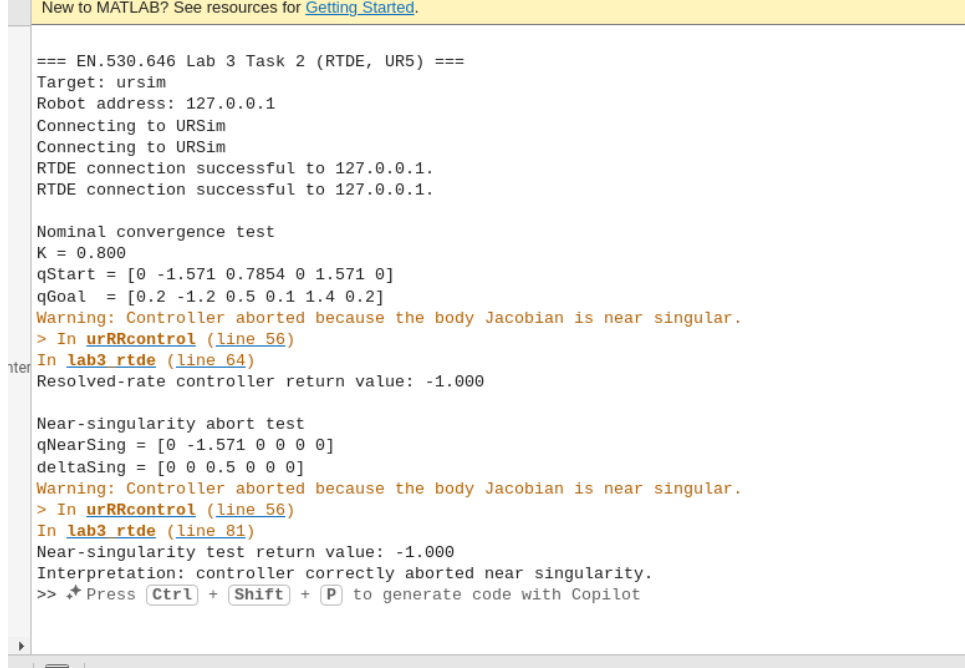
- **Target:** URSim (IP: 127.0.0.1)
- **Robot Model:** UR5
- **Control Gain (K):** 0.800
- **Initialization:** The URSim controller was started, and the RTDE program was loaded and running in the simulation environment before execution.

3.2 Nominal Convergence Test

The nominal experiment aimed to drive the robot from an initial configuration to a target pose $g_{desired}$ derived from q_{goal} .

- **Start Configuration** (q_{start}): $[0, -1.571, 0.7854, 0, 1.571, 0]^T$
- **Target Configuration** (q_{goal}): $[0.2, -1.2, 0.5, 0.1, 1.4, 0.2]^T$

As shown in the terminal output (Figure 5), the controller detected that the body Jacobian was near singular during this specific trajectory, triggered the safety-abort condition, and returned -1.000 . This indicates that the selected q_{goal} or the path towards it approached a singularity, and the controller correctly prioritized safety over convergence.



```

New to MATLAB? See resources for Getting Started.

=== EN.530.646 Lab 3 Task 2 (RTDE, UR5) ===
Target: ursim
Robot address: 127.0.0.1
Connecting to URSim
Connecting to URSim
RTDE connection successful to 127.0.0.1.
RTDE connection successful to 127.0.0.1.

Nominal convergence test
K = 0.800
qStart = [0 -1.571 0.7854 0 1.571 0]
qGoal = [0.2 -1.2 0.5 0.1 1.4 0.2]
Warning: Controller aborted because the body Jacobian is near singular.
> In urRRcontrol (line 56)
In lab3_rtde (line 64)
Resolved-rate controller return value: -1.000

Near-singularity abort test
qNearSing = [0 -1.571 0 0 0 0]
deltaSing = [0 0 0.5 0 0 0]
Warning: Controller aborted because the body Jacobian is near singular.
> In urRRcontrol (line 56)
In lab3_rtde (line 81)
Near-singularity test return value: -1.000
Interpretation: controller correctly aborted near singularity.
>> * Press Ctrl + Shift + P to generate code with Copilot

```

Figure 5: MATLAB Command Window output for RTDE validation. Both tests triggered the singularity abort logic as expected for the given configurations.

3.3 Near-Singularity Abort Test

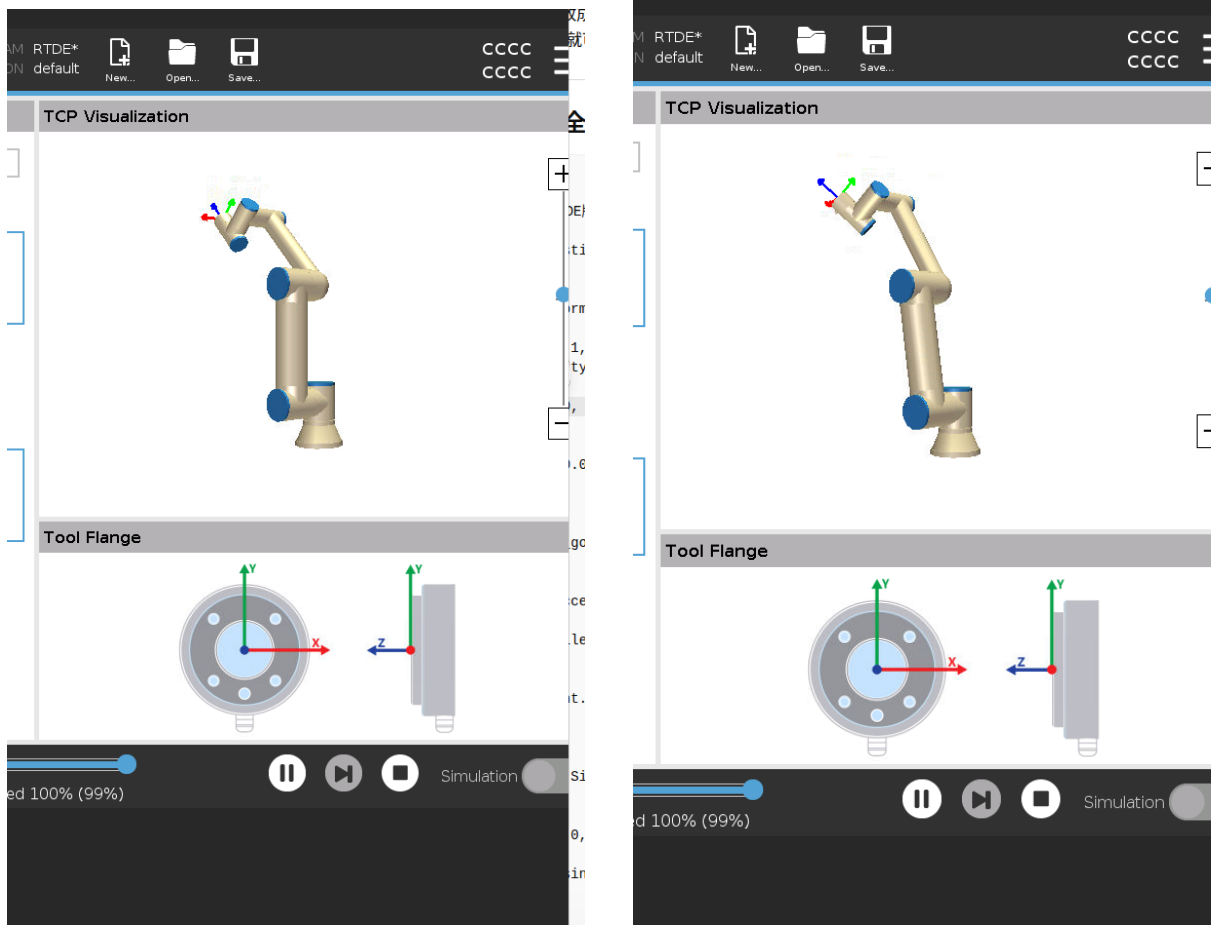
To explicitly verify the singularity detection logic, the robot was initialized at a known singular configuration where $\theta_3 = 0$ and $\theta_5 = 0$:

- **Initial Configuration** ($q_{nearSing}$): $[0, -1.571, 0, 0, 0, 0]^T$
- **Target Displacement** (δ_{sing}): $[0, 0, 0.5, 0, 0, 0]^T$

The controller successfully identified that $\sigma_{min}(J_b) < \text{threshold}$. As shown in Figure 5, the system issued a warning and correctly returned -1.000 . This confirms that the singularity-abort logic is effectively implemented and consistent across both ROS and RTDE interfaces.

3.4 Visualization

The tool-tip motion and configuration in URSim during these tests are visualized below.



(a) Nominal test configuration.

(b) Near-singularity configuration.

Figure 6: URSim TCP and Tool Flange visualization during RTDE validation tests.

4 Team Contributions

In accordance with the lab requirements, the specific contributions of each team member are outlined in the table below:

Table 3: Summary of Team Member Contributions

Member Name	Specific Contributions
Anishka Bhartiya	Task 1: Environment Set-up & Demo
Lujia Yang	Task 2: Core Functions & Report
Xiangzheng Fang	Task 2: Test & Report